Parallel Jules: Git Worktree Swarm Architecture

Status: V1.5.0 (RepoReady)

Author: AI Engineer

Philosophy: Rule of One, VCR, KISS.

1. System Architecture

This workflow implements **Headless Parallel Scripting** using **Git Worktrees** for state isolation. It wraps the Gemini CLI's batch processing capabilities in a Python orchestrator to manage concurrency, conflict resolution, and **OpenTelemetry (OTel) observability**.

The Stack:

- **Execution:** Gemini CLI Headless Mode (--yolo, --output-format stream-json).
- **Isolation:** Git Worktrees (Filesystem containers).
- **Orchestration:** Python asyncio (Process management).
- **Observability:** OpenTelemetry (GCP/Jaeger) + JSONL Event Streaming.

The Lifecycle:

1. **Fan-Out:** Script spawns N worktrees and triggers N headless agents.
2. **Execution:** Agents code in background (no UI) with auto-approval (--yolo).
3. **Monitoring:** Real-time telemetry streams to .jsonl logs and GCP Monitoring.
4. **Fan-In (Serial):** Orchestrator attempts to merge branches one by one.
5. **Guardrails:**
    - **Conflict?** -> Wake Resolver Agent.
    - **Bad Code?** -> Trigger Abort protocol.
    - **Success?** -> Merge & Push.

2. The Orchestrator (swarm_control.py)

The "Manager" that handles the subprocesses, worktrees, and merge queue. Includes hardened git hygiene and logging.

```
import asyncio
import subprocess
import json
import os
import shutil
from dataclasses import dataclass
```

```
from typing import List

# --- CONFIGURATION ---
REPO_ROOT = os.getcwd()
WORKTREE_BASE = os.path.join(REPO_ROOT, ".worktrees")
LOG_DIR = os.path.join(REPO_ROOT, "logs")

@dataclass
class AgentTask:
    name: str
    branch: str
    prompt: str
    skill: str
    tools: List[str]

# Define the Swarm
TASKS = [
    AgentTask(
        name="Agent_A_Auth",
        branch="feat/auth-refactor",
        prompt="Refactor `auth.py` to use Pydantic models. Ensure backward compatibility.",
        skill="feature_architect",
        tools=["fs_read", "fs_write", "terminal_run_test"]
    ),
    AgentTask(
        name="Agent_B_Logger",
        branch="feat/add-logging",
        prompt="Add structured logging to `auth.py` and `database.py`. Use the 'loguru' library.",
        skill="feature_architect",
        tools=["fs_read", "fs_write"]
    )
]

async def setup_environment():
    """Clean slate protocol. Removes old logs, worktrees, and feature branches."""
    print("🧹 [Orchestrator] Cleaning environment...")

    if os.path.exists(LOG_DIR):
        shutil.rmtree(LOG_DIR)
    os.makedirs(LOG_DIR)

    # Prune stale worktrees
    subprocess.run(["git", "worktree", "prune"], check=False)
```

```
    # Force remove worktree directory if it persists
    if os.path.exists(WORKTREE_BASE):
        shutil.rmtree(WORKTREE_BASE)

    # Cleanup branches to prevent 'branch already exists' errors
    for task in TASKS:
        subprocess.run(["git", "branch", "-D", task.branch],
                       stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL)

async def launch_agent(task: AgentTask):
    """Spawns a Jules instance in an isolated Git Worktree."""
    worktree_path = os.path.join(WORKTREE_BASE, task.name)
    # Uses .jsonl extension because we are now streaming events
    log_file = os.path.join(LOG_DIR, f"{task.name}.jsonl")

    print(f"🚀 [Orchestrator] Spawning {task.name} on branch {task.branch}...")

    # 1. Create Worktree
    # Note: 'main' must exist. If using master, change accordingly.
    subprocess.run(["git", "worktree", "add", "-b", task.branch, worktree_path, "main"],
                   check=True, stdout=subprocess.DEVNULL)

    # 2. Construct Gemini Command (Headless Mode)
    cmd = [
        "gemini", "run",
        "--context", f"@.gemini/GEMINI.md @.gemini/skills/{task.skill}.md",
        "--prompt", task.prompt,
        "--tool-strategy", "autonomous",
        # HEADLESS CONFIGURATION
        "--yolo",               # Auto-approve tool use (REQUIRED for background tasks)
        "--output-format", "stream-json", # STREAMING JSON (JSONL) for real-time monitoring
        "--quiet"               # Suppress non-essential stdout
    ]

    # 3. Execute in Background (Isolated CWD)
    with open(log_file, "w") as f:
        process = await asyncio.create_subprocess_exec(
            *cmd,
            cwd=worktree_path,
            stdout=f,
            stderr=f
        )
```

```
        await process.wait()

    return task

async def resolve_conflict(branch: str):
    """The 'Resolver' Agent: Wakes up only for merge conflicts."""
    print(f"⚠️ [Orchestrator] Conflict detected in {branch}. Deploying Resolver...")

    # Updated log name to match Resolver persona
    log_file = os.path.join(LOG_DIR, f"resolver_{branch.replace('/', '_')}.json")

    # We resolve IN the main tree (or a merge staging tree)
    cmd = [
        "gemini", "run",
        # Updated context to git_resolver.md
        "--context", "@.gemini/skills/git_resolver.md",
        "--prompt", f"Resolve git conflicts in current index for branch {branch}. Keep logic from
both HEAD and MERGE_HEAD where possible.",
        "--tool-strategy", "autonomous",
        "--yolo",
        "--output-format", "json" # Resolver uses standard JSON for single-shot response
    ]

    with open(log_file, "w") as f:
        proc = await asyncio.create_subprocess_exec(
            *cmd,
            stdout=f,
            stderr=f
        )
        await proc.wait()

    # Verify resolution
    # 'git diff --check' returns 0 if clean, non-zero if conflict markers remain
    clean_check = subprocess.run(["git", "diff", "--check"], capture_output=True)
    return clean_check.returncode == 0

def validate_code():
    """The Quality Gate: Linting, Black, Tests."""
    print("🛡️ [Orchestrator] Running Quality Gate...")

    # 1. Black Formatting
    subprocess.run(["black", ".", "--check"], stdout=subprocess.DEVNULL)
```

```
    # 2. Tests
    # Capturing output allows us to debug why a merge was rejected if needed
    test_res = subprocess.run(["pytest", "tests/"], capture_output=True)

    return test_res.returncode == 0

async def serial_fan_in(tasks: List[AgentTask]):
    """Merge Queue Logic."""
    print("\n🔄 [Orchestrator] Starting Serial Fan-In...")

    for task in tasks:
        print(f"Trying to merge: {task.branch}")

        # Attempt Merge
        merge_res = subprocess.run(
            ["git", "merge", task.branch, "--no-commit", "--no-ff"],
            capture_output=True
        )

        if merge_res.returncode == 0:
            # Clean merge? Run tests.
            if validate_code():
                subprocess.run(["git", "commit", "-m", f"Merged {task.name}"])
                print(f"✅ Merged {task.branch}")
            else:
                print(f"❌ [ABORT] {task.branch} failed Quality Gate. Reverting.")
                subprocess.run(["git", "merge", "--abort"], stdout=subprocess.DEVNULL)

        else:
            # Conflict!
            resolved = await resolve_conflict(task.branch)
            if resolved and validate_code():
                subprocess.run(["git", "commit", "--no-edit"])
                print(f"✅ [Resolver] Resolved & Merged {task.branch}")
            else:
                print(f"☢️ [NUCLEAR OPTION] Could not resolve {task.branch}. Aborting.")
                subprocess.run(["git", "merge", "--abort"], stdout=subprocess.DEVNULL)

    # Cleanup Worktrees
    # Note: We do not delete the branches here in case manual inspection is needed later
    subprocess.run(["git", "worktree", "remove", "--force"] + [os.path.join(WORKTREE_BASE,
t.name) for t in TASKS],
                   stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL)
```

```
if __name__ == "__main__":
    asyncio.run(setup_environment())
    # Run agents in parallel
    loop = asyncio.get_event_loop()
    loop.run_until_complete(asyncio.gather(*(launch_agent(t) for t in TASKS)))
    # Merge sequentially
    asyncio.run(serial_fan_in(TASKS))
```

3. Agent Contexts (.gemini/)

A. Global Constitution

File: .gemini/GEMINI.md

```
# MISSION
You are a Senior Software Engineer acting as an autonomous worker node.

# CORE PROTOCOLS
1. **Rule of One:** Do one thing well. Do not hallucinate scope.
2. **Environment:** You are running in a **Git Worktree**. You have full control over files in your current directory, but you are isolated from the main branch.
3. **Safety:**
  - NEVER force push.
  - NEVER use `rm -rf` without verifying the path.
  - ALWAYS run local tests before declaring a task complete.
4. **Output:** - When using tools, ensure strict JSON adherence.
  - Be concise. Focus on the `tool_use` payload.
```

B. Skill: Feature Architect (Agents A & B)

File: .gemini/skills/feature_architect.md

```
# SKILL: FEATURE ARCHITECT

## ROLE
Implement robust, production-grade Python code based on specific feature requests.

## WORKFLOW
1. **Discovery:** Read existing code to understand patterns (Type hints, Docstrings).
2. **Implementation:** Write code using `fs_write` (atomic writes).
3. **Verification:**
```

- Create a temporary test file if one does not exist.
- Run `pytest` on your changes.
- Run `black` to format your code.

4. **Commit:** Stage and commit your changes with a conventional commit message (e.g., `feat: ...`).

TOOL RESTRICTIONS
- ALLOW: `fs_read`, `fs_write`, `terminal_run`, `mcp_search`
- DENY: `git_push` (Orchestrator handles sync), `github_pr`

C. Skill: Git Resolver

File: .gemini/skills/git_resolver.md

SKILL: GIT RESOLVER

ROLE
You are the conflict resolution engine. You only wake up when a `git merge` fails.

ALGORITHM
1. **Assess:** Run `git status` and `git diff` to locate "both modified" files.
2. **Parse:** Read files looking for conflict markers:

```text
<<<<<<< HEAD
(Current Main Code)
=======
(Incoming Agent Code)
>>>>>>> branch_name
```

3. **Resolve:**
   - If **Import Conflict**: Keep all unique imports.
   - If **Logic Conflict**: Analyze intent. If Incoming adds logging/tracing, wrap existing logic. If Incoming refactors, prefer Incoming ONLY IF it preserves behavior.
   - **Safety**: Do not delete existing business logic (VCR check).
4. **Finalize:** - Write the clean file.
   - Run git add <file>.
   - **DO NOT COMMIT**. The Orchestrator will commit after verification.

4. Safety & Tools

A. MCP Configuration & Allow-List
`File: .gemini/settings.json`

```json
{
  "mcpServers": {
    "local-fs": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-filesystem", "./"]
    },
    "python-tools": {
      "command": "uv",
      "args": ["run", "mcp-server-python"]
    }
  },
  "toolConfig": {
    "allowList": [
      "fs_read_file", "fs_write_file", "fs_list_directory",
      "terminal_run_command"
    ],
    "denyList": [
      "fs_delete_directory", "github_create_issue"
    ]
  },
  "telemetry": {
    "enabled": true,
    "target": "gcp"
  },
  "extensions": {
    "hooks": "./.gemini/hooks.js"
  }
}
```

B. Runtime Safety Hooks (The YOLO Guard)

File: .gemini/hooks.js

This Javascript hook acts as the **Programmatic Veto**. Since --yolo disables human approval, this script takes over the responsibility of blocking dangerous actions.

SECURITY UPDATE: Uses path.resolve to prevent directory traversal escapes.

```
const path = require('path');

module.exports = {
  /**
```

```
   * Pre-execution guardrail.
   * Prevents agents from accessing sensitive directories or running destructive commands.
   */
  onBeforeToolExecution: async (toolName, args, context) => {

    // 1. Prevent escaping the Worktree (Sandbox Enforcer)
    // We strictly resolve paths to ensure agents can't use "../../" to break out.
    if (args.path) {
        const currentDir = process.cwd();
        const resolvedPath = path.resolve(currentDir, args.path);

        // If the Worktree logic is working, currentDir should be inside .worktrees/
        // But we explicitly check that the target is still inside the current CWD.
        if (!resolvedPath.startsWith(currentDir)) {
            throw new Error(`⛔ SECURITY: Directory traversal blocked. Agent trapped in:
${currentDir}`);
        }
    }

    // 2. Block High-Risk Terminal Commands
    if (toolName === 'terminal_run_command') {
      const forbidden = ['rm -rf /', 'git push', 'git reset --hard', 'shutdown', 'mkfs'];
      if (forbidden.some(cmd => args.command.includes(cmd))) {
        throw new Error(`⛔ SECURITY: Command '${args.command}' is blacklisted.`);
      }
    }

    // 3. Log for Orchestrator (Structured Audit)
    // This feeds into the stream-json output for the Orchestrator
    console.log(JSON.stringify({
      timestamp: new Date().toISOString(),
      level: "AUDIT",
      tool: toolName,
      args: args
    }));
  }
};
```

5. Unit Tests

A. Orchestrator Logic (tests/test_swarm.py)

Run via pytest. Mocks the actual OS/Git calls to verify logic safely.

```
import pytest
from unittest.mock import patch, MagicMock, AsyncMock
import swarm_control

@pytest.mark.asyncio
async def test_launch_agent_constructs_correct_command():
    """Verify that agents are launched with the critical headless/yolo flags."""
    task = swarm_control.AgentTask(
        name="TestAgent",
        branch="feat/test",
        prompt="Do work",
        skill="test_skill",
        tools=[]
    )

    # Mock asyncio subprocess creation
    with patch("asyncio.create_subprocess_exec", new_callable=AsyncMock) as mock_exec:
        mock_process = AsyncMock()
        mock_process.wait.return_value = None
        mock_exec.return_value = mock_process

        # Mock file opening and git execution
        with patch("builtins.open", new_callable=MagicMock), \
             patch("subprocess.run"):

            await swarm_control.launch_agent(task)

            # Assertions
            args, _ = mock_exec.call_args
            cmd = list(args)

            # VCR: These flags are non-negotiable for headless ops
            assert "gemini" in cmd
            assert "--yolo" in cmd, "Agent must run in YOLO mode for background execution"
            assert "--output-format" in cmd
            assert "stream-json" in cmd, "Output must be streaming JSON for logging"
            assert f"@.gemini/skills/{task.skill}.md" in cmd[cmd.index("--context") + 1]

@pytest.mark.asyncio
async def test_resolver_invocation():
    """Verify the Resolver is called with the correct specialized skill."""
```

```
    with patch("asyncio.create_subprocess_exec", new_callable=AsyncMock) as mock_exec:
        mock_process = AsyncMock()
        mock_process.wait.return_value = None
        mock_exec.return_value = mock_process

        with patch("builtins.open", new_callable=MagicMock):
            await swarm_control.resolve_conflict("test/branch")

            args, _ = mock_exec.call_args
            cmd = list(args)

            assert "@.gemini/skills/git_resolver.md" in cmd[cmd.index("--context") + 1]
```

B. Security Hook Logic (tests/test_hooks.js)

Run via node tests/test_hooks.js. Validates the Sandbox path resolution.

```
const assert = require('assert');
const hooks = require('../.gemini/hooks.js');
const path = require('path');

async function testSandbox() {
    console.log("🛡️ Testing Hook Security...");

    const mockContext = {};
    const safePath = 'safe_file.txt';
    const attackPath = '../../etc/passwd';

    // 1. Valid Path
    try {
        await hooks.onBeforeToolExecution('fs_read', { path: safePath }, mockContext);
        console.log("✅ Safe path allowed.");
    } catch (e) {
        console.error("❌ False positive on safe path:", e);
    }

    // 2. Attack Path
    try {
        await hooks.onBeforeToolExecution('fs_read', { path: attackPath }, mockContext);
        console.error("❌ FAILED: Attack path was allowed!");
    } catch (e) {
        if (e.message.includes("SECURITY")) {
```

```
        console.log("✅ Attack path blocked.");
      } else {
        console.error("❌ Unexpected error:", e);
      }
    }
}

testSandbox();
```

6. Usage (README)

Parallel Jules Swarm

An agentic orchestration layer for parallel feature development using Gemini CLI and Git Worktrees.

Prerequisites

- **Gemini CLI:** v2.0+ (Installed & Authenticated)
- **Python:** 3.10+
- **NodeJS:** 18+ (For MCP/Hooks)
- **Git:** 2.20+

Quick Start

1. **Install Dependencies:**
   `pip install black pytest loguru`
   `npm install @modelcontextprotocol/server-filesystem`

2. **Configure Tasks:** Edit `TASKS` in `swarm_control.py` to define the agents you want to spin up.
3. **Launch Swarm:**
   `python swarm_control.py`

4. **Monitor:** In a separate terminal, watch the thoughts of a specific agent:
   `tail -f logs/Agent_A_Auth.jsonl`

Architecture Flow

Orchestrator -> Git Worktree (A) -> Agent A (Gemini) -> Hooks (Safety) -> MCP (Tools) Git Worktree (B) -> Agent B (Gemini) ...

Emergency Stop

If an agent goes rogue (e.g., infinite loop):

1. **Kill Python:** Ctrl+C in the swarm_control.py terminal.
2. **Kill Background Processes:**
   pkill -f "gemini run"

3. **Cleanup Worktrees:**
   git worktree prune
   rm -rf .worktrees/